

CZOI Toolkit — User Guide

Version 1.1 · A guide to building secure, intelligent, and integrated organizational systems with the Constrained Zoned-Object Architecture

Table of Contents

1. [Introduction](#)
 2. [Installation](#)
 3. [Your First CZOI System](#)
 4. [Core Concepts](#)
 5. [Building Zone Hierarchies](#)
 6. [Roles, Users, and Operations](#)
 7. [The Permission Calculus](#)
 8. [Constraints with UniLog](#)
 9. [Daemons and Signals](#)
 10. [Neural Components](#)
 11. [Semantic Embeddings](#)
 12. [Adaptive Access Control](#)
 13. [Audit and Observability](#)
 14. [Integrating Heterogeneous Systems](#)
 15. [Persistence and Web Frameworks](#)
 16. [Complete Worked Examples](#)
 17. [Cookbook](#)
 18. [Troubleshooting](#)
 19. [Best Practices](#)
 20. [API Reference](#)
-

1. Introduction

What is CZOI?

The **CZOI toolkit** is the reference Python implementation of the **Constrained Zoned-Object Architecture (CZOA)** — a unified framework for building systems that are simultaneously:

- **Secure** — enforce strict, auditable access policies.
- **Intelligent** — adapt to changing conditions through neural components.
- **Organisation-aligned** — mirror the hierarchical structure of real enterprises.
- **Integrable** — compose heterogeneous subsystems into a coherent federation without sacrificing local autonomy.

The core idea is simple but powerful: **every organizational unit is a zone, and every zone is itself a full system**. A hospital is a zone. So is its Emergency department. So is a specific bed in the Emergency department. And so is a legacy mainframe wrapped in an adapter zone — a 20-year-old system that participates in the federation as a first-class citizen without being modified.

Each zone has its own roles, operations, constraints, daemons, and neural components. Each inherits from its parent. Each can be developed, tested, and deployed independently. And each can be composed with its siblings under shared governance.

Three fragmentations, one solution

Most modern organizations are not coherent systems — they are archipelagos of systems. Three fragmentations block coherent operation:

1. **Intelligence vs. security.** AI models and access-control systems have evolved separately, with different abstractions, correctness criteria, and timescales.
2. **Local autonomy vs. global governance.** Subunits want to evolve at their own rate; governance wants a single policy. Traditional architectures force a choice.
3. **Heterogeneous systems vs. coherent operations.** Real organizations run on dozens of incompatible systems. Making them work together is the central challenge of modern enterprise IT.

CZOA addresses all three with a single mechanism: recursive composition. Adapter zones integrate legacy systems without modification. The recursive permission calculus provides global integration guarantees without collapsing local autonomy. And neural components live inside every zone, so intelligence is preserved across subsystem boundaries.

Who is this guide for?

Software engineers, system architects, integration architects, and researchers who need to build:

- Enterprise access-control systems with formal security guarantees.
- Adaptive systems that respond to changing conditions without violating policy.
- System-of-systems architectures where each division retains local autonomy under global governance.
- **Post-merger IT landscapes** that must unify without rewriting legacy systems.
- **Multi-tenant platforms** where tenants need isolation and shared services.
- **Federated consortia** where independent organizations must interoperate under shared rules.

What you need to know

- **Python:** comfort with classes, decorators, and async ([asyncio](#)).
- **Basics of RBAC:** what roles and permissions are.
- No prior knowledge of CZOA, UniLog, or category theory is required.

Reading this guide

Sections 3–7 are the **core tutorial** — read them in order. Sections 8–13 are **feature guides** you can dip into as needed. **Section 14 is the integration guide** — read it if you are federating heterogeneous systems. Sections 16–20 are **reference** material.

Every code snippet in this guide is self-contained and runnable.

2. Installation

Requirements

- Python 3.9 or later
- The `unilog-toolkit` package (installed automatically)

Standard install

```
pip install czoι-toolkit
```

With optional features

```
pip install czoι-toolkit[neural]      # scikit-learn for RoleMiner
pip install czoι-toolkit[embedding]  # sentence-transformers for semantic
embeddings
pip install czoι-toolkit[dev]        # pytest, ruff, mypy
```

Verify the installation

```
import czoι
print(czoι.__version__)  # 1.1.0
```

If this prints without error, you're ready. If you see `UniLogBridgeError` later, check that `unilog-toolkit` is installed:

```
pip show unilog-toolkit
```

From source

```
git clone https://github.com/hongxueharriswang/czoι-toolkit.git
cd czoι-toolkit
pip install -e ".[dev,neural,embedding]"
```

3. Your First CZOI System

Let's build the simplest possible CZOI system: a company with an HR department, two roles, and a permission check.

```
from czoι import (
    Application, CZOABuilder, Decision, Operation, Role, User,
)
```

```

# -----
# 1. Create the system and its zones
# -----
builder = CZOABuilder("AcmeCorp")
company = builder.root
hr      = builder.add_zone("HR", parent=company, atomic=True)

# -----
# 2. Define an application and its operations
# -----
hr_app = Application("HRSysTem", zone=hr)
view_employee = hr_app.add_operation(Operation("view_employee"))
edit_employee = hr_app.add_operation(Operation("edit_employee"))
hr.add_application(hr_app)

# -----
# 3. Create roles
# -----
manager = Role("Manager", zone=hr,
               base_permissions=[view_employee, edit_employee])
assistant = Role("Assistant", zone=hr,
                 base_permissions=[view_employee])
manager.add_junior(assistant) # Manager is senior to Assistant

hr.add_role(manager)
hr.add_role(assistant)

# -----
# 4. Create users (containment principle enforced automatically)
# -----
alice = User("alice", roles={"Assistant"})
bob   = User("bob",   roles={"Manager"})

company.add_user(alice)      # register at root first
hr.add_user(alice)           # then at the child zone

company.add_user(bob)
hr.add_user(bob)

# -----
# 5. Check permissions
# -----
engine = builder.permission_engine

print(engine.decide(alice, view_employee, hr).name) # ALLOW
print(engine.decide(alice, edit_employee, hr).name) # INCONCLUSIVE
print(engine.decide(bob,   edit_employee, hr).name) # ALLOW

```

What just happened?

1. **CZOABuilder** creates the root zone and wires together a permission engine, constraint manager, daemon manager, and embedding service. Every zone you add inherits these automatically.

2. `Role.add_junior()` establishes the intra-zone seniority relation. Because Manager is senior to Assistant, Bob inherits all of Alice's permissions on top of his own.
3. `zone.add_user()` enforces the containment principle: a user must be registered at the parent before any child can hold them. This is what makes the hierarchy coherent.
4. `engine.decide()` returns a three-valued `Decision`:
 - `ALLOW` — the user has a covering role in this zone.
 - `DENY` — the user has a covering role but a constraint forbids it.
 - `INCONCLUSIVE` — no covering role found locally; a parent might override.

Integration preview

In a federation, this same `decide` call transparently spans subsystem boundaries. A user in one subsystem requesting an operation in another subsystem flows through both subsystems' local decision functions, and the more restrictive constraint wins. Section 14 shows how to wire this up with adapter zones and the federation builder.

4. Core Concepts

The 10-tuple

Every CZOI zone is a 10-tuple:

S = (Z, R, U, A, O, N, E, Γ, Φ, Δ)

Symbol	Name	What it holds	Integration role
Z	Zones	Child subsystems (recursive)	Composition operator
R	Roles	Job functions with base permissions	Cross-system vocabulary
U	Users	Identities with roles and attributes	Federated identity
A	Applications	Structural modules (deployable units)	Origin signal for embeddings
O	Operations	Atomic executable actions	Atomic interface contract
N	Neural	Trainable functions (predictors, detectors)	Local models, no global retraining
E	Embeddings	Semantic vectors + alignment	Vocabulary bridge across silos
Γ	Constraints	Identity, trigger, goal, access rules	Shared governance layer
Φ	Permissions	Two-stage recursive decision function	Cross-boundary decision flow
Δ	Daemons	Continuous monitoring processes	Cross-zone health monitoring

These ten components are **orthogonal**: each has a distinct role and none of them overlaps with another. This is what makes CZOI minimal — and what makes it *composable*. If two components did the same job, you'd

have to decide which one to use in each context, and the system would have internal ambiguity that breaks under composition.

Three kinds of zone

CZOI provides two native zone types plus one integration-specific type:

- **CompositeZone** (the default) — may contain child zones. `Z_z` may be non-empty.
- **AtomicZone** — a leaf. Calling `add_zone` on it raises `TypeError`. `Z_z = ∅`.
- **AdapterZone** — a leaf whose internal behaviour is opaque, wrapping a legacy or external system through a declared interface. The mechanism for integrating systems that were not built with CZOI in mind.

```
# Explicit composite
hospital = builder.add_zone("Hospital")           # CompositeZone

# Explicit atomic
emergency = builder.add_zone("Emergency", parent=hospital, atomic=True)

# Adapter zone wrapping a legacy system
legacy_hr = AdapterZone(name="LegacyHR", adapter=MyHRAdapter())
```

Applications vs Operations

This is the most important design decision in CZOI:

- An **application** is a *structural module*. It groups related operations and is the natural unit of deployment.
- An **operation** is an *atomic permission target*. It is the only thing you can grant a role.

```
app = Application("Payroll", zone=hr)
process = app.add_operation(Operation("process_payroll"))
adjust = app.add_operation(Operation("adjust_salary"))
hr.add_application(app)

# You grant permissions on OPERATIONS, not on applications:
admin_role.grant(process)      # correct
admin_role.grant(adjust)      # correct
# admin_role.grant(app)       # WRONG — will raise or misbehave
```

Why this matters:

- **Minimality**: if applications were permission targets, you'd need two parallel permission systems — one for apps and one for ops. CZOI avoids that.
- **Granularity**: you can grant a role the ability to `view_payroll` without granting `edit_payroll`, even if they're in the same application.
- **Auditing**: every access log entry names a specific operation, not a vague "used the Payroll app."

- **Integration:** operations are the atomic interface contract between subsystems. Two subsystems are integrable at the operation level whenever their operations can be mapped to each other.

5. Building Zone Hierarchies

Creating the tree

```
builder = CZOABuilder("University")

uni      = builder.root
coe      = builder.add_zone("CollegeOfEngineering", parent=uni)
cs       = builder.add_zone("CSDepartment", parent=coe, atomic=True)
registrar = builder.add_zone("Registrar", parent=uni, atomic=True)
```

Result:

```
University (composite)
├── CollegeOfEngineering (composite)
│   └── CSDepartment (atomic)
└── Registrar (atomic)
```

Walking the tree

```
for zone in builder.root.walk():
    print(f"{' ' * zone.depth()}- {zone.name}")

# University
# - CollegeOfEngineering
#   - CSDepartment
# - Registrar
```

The containment principle

Every child zone's user set is a subset of its parent's:

$$U_{\text{child}} \subseteq U_{\text{parent}}$$

This is enforced automatically. When you register a user:

```
user = User("alice", roles={"Student"})

uni.add_user(alice)          # OK — root
```

```
coe.add_user(alice)          # OK – parent already has alice
cs.add_user(alice)           # OK – parent already has alice

# Attempting to register at a child before the parent raises:
# cs.add_user(other_user)    # raises ZoneContainmentError
```

Containment and integration

Containment is what makes shared governance tractable in a federation: the merged authority sees every user in every subsystem, but each subsystem sees only its own users. Authority flows downward; autonomy is preserved upward. This is the structural reason why CZOI federations can impose shared policy without intruding into how each subsystem operates internally.

Ancestry and depth

```
cs.ancestry()                # [University, CollegeOfEngineering, CSDepartment]
cs.depth()                   # 2
cs.is_ancestor_of(cs)        # True
uni.is_ancestor_of(cs)       # True
```

Properties on zones

Every zone has a typed property store:

```
cs.properties.set("capacity", 500, type_hint="int")
cs.properties.set("building", "Turing Hall", type_hint="str")

cs.properties.get("capacity")          # 500
cs.properties.get("building")          # "Turing Hall"
cs.properties.get("missing", default=0) # 0
```

The type hint enforces correctness:

```
cs.properties.set("capacity", "many", type_hint="int")
# TypeError: capacity: expected int, got <class 'str'>
```

Properties are visible to constraints and daemons — they are the "state" of your system.

6. Roles, Users, and Operations

Creating roles

```

doctor = Role(
    "Doctor",
    zone=hospital,
    base_permissions=[prescribe, view_patient, order_lab],
)
hospital.add_role(doctor)

```

A role always belongs to a specific zone. It is inherited by all descendant zones automatically:

```

# If Hospital has child zone Emergency:
emergency.roles["Doctor"] is doctor    # True – same role object

```

Seniority (intra-zone inheritance)

```

attending = Role("Attending", zone=hospital, base_permissions=[prescribe])
resident = Role("Resident", zone=hospital, base_permissions=[view_patient])

attending.add_junior(resident)
# Attending now inherits everything Resident can do, plus prescribe.

```

You can chain seniority:

```

chief.add_junior(attending)    # Chief inherits Attending's permissions
# Chief now transitively inherits Resident's permissions too.

```

Granting and revoking permissions

```

# Direct (fast, no cache invalidation)
role.grant(operation)
role.revoke(operation)

# Through the zone (invalidates the permission engine's cache)
hospital.grant(role, operation)
hospital.revoke(role, operation)

```

Prefer the zone-level methods whenever you're in a running system — they keep the permission cache consistent.

Creating users

```
alice = User(
    "alice",
    roles={"Doctor"},
    attributes={
        "employee_id": "E-1234",
        "shift": "day",
        "specialty": "cardiology",
    },
    credentials={"password_hash": "..."},
)
```

Users carry:

- **roles** — a set of role *names*. Zones resolve them locally.
- **attributes** — a typed property store, like zone properties.
- **credentials** — anything you want (password hash, tokens, etc.).

```
alice.roles                # {"Doctor"}
alice.has_role("Doctor")   # True
alice.attributes.get("specialty") # "cardiology"
alice.attributes.set("on_call", True)
```

Registering users

```
# Always register along the containment path
for zone in [root, hospital, emergency]:
    zone.add_user(alice)
```

Or use a helper:

```
def register_along_path(user, zone):
    for z in zone.ancestry():
        if user.name not in z.users:
            z.add_user(user)

register_along_path(alice, emergency) # adds to Emergency, Hospital, Root
```

Role taxonomies and integration

When two subsystems come from different origins — a merger, an acquisition, a partner API — their role taxonomies are typically distinct. CZOI preserves both: each subsystem keeps its own roles internally, and the federation discovers semantic equivalence between roles across subsystems using the embedding alignment functor. Section 14 shows how this works in practice.

7. The Permission Calculus

Basic decisions

```
from czoil import Decision

decision = engine.decide(user, operation, zone)
# Decision.ALLOW / Decision.DENY / Decision.INCONCLUSIVE
```

Three values, not two:

- **ALLOW** — the user has a covering role here.
- **DENY** — the user has a covering role, but a constraint forbids the operation.
- **INCONCLUSIVE** — no covering role in this zone. A parent might still grant.

The recursive structure

When you call `decide(user, op, zone)`:

1. The engine checks the user's roles **in this zone**.
2. If a role covers the operation, it evaluates the zone's access constraints.
3. If constraints pass → **ALLOW**.
4. If a role covers but constraints fail → **DENY**.
5. If no role covers → the engine recurses to the parent zone.
6. If the root is reached and still inconclusive → **DENY**.

```
# Suppose only Hospital has the Doctor role, and the user is registered
# in Emergency (a child of Hospital). The engine will:
# 1. Look in Emergency → no Doctor role → INCONCLUSIVE
# 2. Recurse to Hospital → Doctor role found → ALLOW
print(engine.decide(alice, prescribe, emergency).name) # ALLOW
```

Effective permissions

The engine computes the user's effective permission set as:

```
P_effective(role, zone) =
  P_base(role)
  ∪ ⋃_{r' ∈ junior(role)} P_base(r')
  ∪ ⋃_{γ : (zone, role) → (zone', role')} P_base(role')
```

In code, this is:

```
def effective(role):
    ops = set(role.base_permissions)
```

```
for junior in role.junior_roles:
    ops |= effective(junior)    # recursive
return ops
```

You can inspect it directly:

```
perms = {op.qualified_name for op in effective(attendng)}
print(perms)
# {'EMR.prescribe', 'EMR.view_patient', 'EMR.order_lab'}
```

Caching

The engine caches decisions by `(user, operation, zone)`. This gives sub-millisecond lookups in production:

```
print(engine.stats())
# {'hits': 43, 'misses': 1180, 'parent_lookups': 4, 'denies': 0}
```

The cache is invalidated automatically when you:

- Call `zone.grant(role, op)` or `zone.revoke(role, op)`.
- Add a new role to a zone.
- Add a new user to a zone.
- Register a new operation.

If you mutate a role's `base_permissions` directly (via `role.grant(op)`), the cache is **not** invalidated. Prefer the zone-level methods.

Enabling audit

```
from czoι import PermissionEngine

builder.permission_engine = PermissionEngine(
    cache_enabled=True,
    audit_enabled=True,
)
for zone in builder.root.walk():
    zone.set_permission_engine(builder.permission_engine)

# ... run some decisions ...

for record in builder.permission_engine.audit[-5:]:
    print(f"{record.timestamp} {record.user} "
          f"{record.operation} @ {record.zone} → {record.decision.name}")
```

Each **DecisionRecord** has:

Field	Meaning
timestamp	UTC datetime
user	User name
operation	Qualified operation name
zone	Zone name
decision	Decision enum
stage	"local", "parent-override", "root-deny", or "cache"

Cross-zone decisions

In a federation, a decision for a user in subsystem A requesting an operation in subsystem B traverses both subsystems' local decision functions. The audit record shows the full path — including which subsystem imposed which constraint. This is what makes cross-subsystem compliance auditable, not just observable.

8. Constraints with UniLog

CZOA defines four families of constraints — $\Gamma = (\mathbf{I}, \mathbf{T}, \mathbf{G}, \mathbf{C})$:

- **I** — Identity: invariants that must always hold.
- **T** — Trigger: event-condition-action rules.
- **G** — Goal: optimisation objectives.
- **C** — Access: permission-related rules.

The toolkit delegates constraint specification to the **UniLog toolkit**, a fibred logic framework with eleven solvers.

Adding a constraint

```
builder.add_access_constraint("""
    signature {
        sort User, Role;
        constant Doctor : Role;
        constant Auditor : Role;
        predicate hasRole(u: User, r: Role);
    }
    forall u: User .
        not (hasRole(u, Doctor) and hasRole(u, Auditor))
""")
```

The syntax is UniLang — first-order logic with temporal, modal, and deontic operators. The **signature** block declares the sorts, constants, functions, and predicates you'll use. Everything after is the constraint

itself.

The four kinds

```
# Identity – an invariant
builder.add_identity_constraint("""
    signature { sort Zone; predicate valid(z: Zone); }
    forall z: Zone . valid(z) -> valid(z)
""")

# Trigger – event-condition-action
builder.add_trigger_constraint("""
    signature {
        sort Patient;
        predicate critical(p: Patient);
        predicate alerted(p: Patient);
    }
    forall p: Patient . critical(p) -> alerted(p)
""")

# Goal – optimisation objective
builder.add_goal_constraint("""
    signature {
        sort KPI;
        predicate minimised(k: KPI);
        predicate optimised(k: KPI);
    }
    forall k: KPI . minimised(k) -> optimised(k)
""")

# Access – permission policy
builder.add_access_constraint("""
    signature {
        sort User;
        predicate prescribe(u: User);
        predicate dispense(u: User);
    }
    forall u: User . not (prescribe(u) and dispense(u))
""")
```

Built-in predicates

The `CZOIModel` automatically exposes:

Predicate	Meaning
<code>inZone(u, z)</code>	User <code>u</code> is registered in zone <code>z</code>
<code>childOf(c, p)</code>	Zone <code>c</code> is a direct child of zone <code>p</code>
<code>hasRole(u, r)</code>	User <code>u</code> holds role <code>r</code>

Predicate	Meaning
<code>canPerform(u, o)</code>	User <code>u</code> is allowed to perform operation <code>o</code>

And one built-in function: `parent(z)` — the parent zone of `z` (or `z` itself for the root).

Custom predicates

```
builder.register_predicate(
    "onShift",
    lambda u: u.attributes.get("shift") == "day",
)

builder.register_predicate(
    "zoneFull",
    lambda z: len(z.users) >= z.properties.get("capacity", 999),
)

builder.add_access_constraint("""
    signature {
        sort User, Zone;
        predicate onShift(u: User);
        predicate zoneFull(z: Zone);
        predicate inZone(u: User, z: Zone);
    }
    forall u: User, z: Zone .
        (inZone(u, z) and zoneFull(z) and not onShift(u))
        -> not inZone(u, z)
""")
```

Checking constraints manually

```
results = builder.constraint_manager.check_all()
for kind, ok in results.items():
    print(f"{kind}: {'OK' if ok else 'VIOLATED'}")

# Or against a specific zone:
results = builder.constraint_manager.check_zone(emergency)
```

Constraint evaluation hooks

Access constraints (`C` kind) are evaluated automatically inside the permission engine. When you call `engine.decide(...)`, the engine:

1. Finds a covering role.
2. Calls `constraint_manager.is_satisfied(user, op, zone)`.
3. If any access constraint fails → `DENY`.

This is why you get **DENY** vs **INCONCLUSIVE**: the former means a constraint blocked you; the latter means no role covered you.

Shared constraints in a federation

When subsystems are federated, the parent can impose **shared constraints** that apply to every subsystem. A child can add stricter local constraints, but cannot weaken a parent's. This is the formal mechanism by which shared governance is imposed on independent actors without violating their internal policies — and it is a one-line operation on the federation builder:

```
fed.share_constraint("""... UniLang ...""")
```

9. Daemons and Signals

Daemons (Δ) are continuous monitors. They run periodically, read the system state, and emit typed signals up a daemon tree.

The Daemon base class

```
from czoι import Daemon, DaemonSignal

class BatteryDaemon(Daemon):
    def __init__(self, robots, threshold=0.2, parent=None):
        super().__init__("BatteryDaemon", parent=parent, interval=1.0)
        self.robots = robots
        self.threshold = threshold

    def monitor(self):
        for r in self.robots:
            batt = r.attributes.get("battery", 1.0)
            if batt < self.threshold:
                self.emit_signal(
                    DaemonSignal.STATE_WARNING,
                    {"robot": r.name, "battery": batt},
                )
```

The `monitor()` method is called periodically by the `DaemonManager`. It should be synchronous (no `async`).

The signal hierarchy

```
class FleetDaemon(Daemon):
    def __init__(self, parent=None):
        super().__init__("FleetDaemon", parent=parent, interval=2.0)
        self.warnings = []
        self.criticals = []
```

```
def on_signal(self, signal, payload, source=None):
    if signal is DaemonSignal.STATE_WARNING:
        self.warnings.append((source.name, payload))
    elif signal is DaemonSignal.STATE_CRITICAL:
        self.criticals.append((source.name, payload))
```

The default `handle_signal` calls `on_signal` locally, then propagates to `self.parent`. This gives you a natural tree of escalation:

```
FleetDaemon (root)
├── BatteryDaemon      – emits STATE_WARNING on low battery
├── OvercurrentDaemon – emits STATE_CRITICAL on overcurrent
└── ReportDaemon      – emits informational signals
```

Attaching daemons

```
fleet = FleetDaemon()
battery = BatteryDaemon(robots, parent=fleet)
overcurrent = OvercurrentDaemon(zones, parent=fleet)

for d in (fleet, battery, overcurrent):
    builder.add_daemon(d)
```

`builder.add_daemon()` registers the daemon with the root zone **and** with the `DaemonManager` — you don't need to do both.

Running daemons

Synchronous — for tests and step-based simulations:

```
for step in range(60):
    sim.step()
    builder.daemon_manager.tick()    # invokes every monitor once
```

Asynchronous — for production:

```
import asyncio

async def main():
    await builder.daemon_manager.run(duration=300.0)

asyncio.run(main())
```

The `DaemonManager` runs each daemon's `monitor()` in a thread from a `ThreadPoolExecutor`, so a slow monitor never blocks the event loop.

Per-daemon intervals

```
BatteryDaemon(robots, parent=fleet, interval=0.5) # every 500 ms
OvercurrentDaemon(zones, parent=fleet, interval=2.0) # every 2 s
```

Error isolation

```
def safe_monitor(self):
    if not self.enabled:
        return
    try:
        self.monitor()
    except Exception:
        log.exception("Daemon %s raised", self.name)
```

The `DaemonManager` always calls `safe_monitor`, never `monitor` directly.

Cross-zone daemons

In a federation, **cross-zone daemons** observe pairs of subsystems and emit into their common ancestor. They detect integration issues before they become incidents: vocabulary drift (a subsystem's roles gradually diverge from the shared taxonomy), latency spikes in cross-zone flows, and anomalous interaction patterns. Section 14 shows how to wire these up.

10. Neural Components

The toolkit ships with three neural primitives plus a wrapper for arbitrary models.

Predictor

```
from czoι import Predictor
import numpy as np

# Option 1: fit from data
predictor = Predictor("sepsis", threshold=0.85)
X = np.array([...]) # shape (n, features)
y = np.array([...]) # shape (n,), values in {0, 1}
predictor.fit(X, y, epochs=500, lr=0.05)

predictor.predict({"hr": 0.8, "temp": 0.9, "lactate": 0.7})
# -> float in [0, 1]

predictor.fires({"hr": 0.8, "temp": 0.9, "lactate": 0.7})
```

```
# -> bool (predict >= threshold)

# Option 2: wrap an existing function
predictor = Predictor("rule", fn=lambda f: 1.0 if f["x"] > 5 else 0.0)
```

AnomalyDetector

```
from czoι import AnomalyDetector
import numpy as np

# 1. Collect normal samples
X_normal = np.random.normal(0, 1, size=(1000, 5))

# 2. Build and fit the detector
detector = AnomalyDetector(
    name="access_anomaly",
    input_dim=5,
    latent_dim=3,
    threshold=0.5,      # will be recalibrated in fit()
    seed=42,
)
detector.fit(X_normal, epochs=300, lr=0.05)

# 3. Score new samples
sample = np.array([10.0, 10.0, 10.0, 10.0, 10.0])
detector.score(sample)      # float (reconstruction error)
detector.is_anomalous(sample) # bool (score > calibrated threshold)
```

After `fit`, `detector.threshold` is set to the 95th percentile of training scores.

⚠ Important

Always z-score normalise your features before fitting. The autoencoder uses tanh activations; unscaled features dominate the loss.

```
mean = X_normal.mean(axis=0)
std = X_normal.std(axis=0)
X_normal_norm = (X_normal - mean) / std
```

RoleMiner

```
from czoι import RoleMiner
import numpy as np

# Binary matrix: rows = users, columns = operations
X = np.array([
    [1, 1, 0, 0],
```

```

    [1, 1, 0, 0],
    [0, 0, 1, 1],
    [0, 0, 1, 1],
], dtype=float)

op_names = ["A.read", "A.write", "B.read", "B.write"]

miner = RoleMiner(latent_dim=2, min_cluster_size=2, seed=42)
result = miner.mine(X, op_names, min_support=0.5)

print(result.suggested_roles)
# {'MinedRole_0': ['A.read', 'A.write'],
#  'MinedRole_1': ['B.read', 'B.write']}
print(result.confidence)
# {'MinedRole_0': 1.0, 'MinedRole_1': 1.0}
print(result.n_clusters)    # 2

```

Custom neural components

```

class MyModel:
    def predict(self, features) -> float:
        ...

zone.add_neural("my_model", MyModel())

```

You can then reference it in daemons, constraint predicates, or a [NeuralContribution](#):

```

def hook(user, operation, zone, base):
    model = zone.neural.get("my_model")
    if model and model.predict({"user": user.name}) > 0.9:
        return Decision.DENY
    return base

```

Local intelligence in federations

In a federation, neural components are *strictly local*. Each subsystem keeps its own models, trained on its own data, without any requirement to share them or re-train them globally. The federation composes their *outputs*, not their *state*. This is what preserves local autonomy while enabling shared governance — each subsystem's intelligence remains its own, even as the federation's policy applies to all.

11. Semantic Embeddings

The [EmbeddingService](#) maps entities (operations, roles, zones) to vectors in a Hilbert space so that semantic similarity becomes geometric proximity.

Basic usage

```
from czoι import EmbeddingService

emb = EmbeddingService(dimension=64) # hash-based by default

v_op    = emb.embed_operation(operation)
v_role  = emb.embed_role(role)

similarity = emb.similarity(v_op, v_role)
```

With transformer backend

```
emb = EmbeddingService(
    model_name="all-MiniLM-L6-v2",
    use_transformer=True,
)
```

Now embeddings capture real semantic content:

```
v1 = emb.embed("patient_discharge_summary")
v2 = emb.embed("clinical_discharge_note")
v3 = emb.embed("payroll_processing")

emb.similarity(v1, v2) # high – both are discharge documents
emb.similarity(v1, v3) # low – unrelated domains
```

Global alignment functor

```
v_local = emb.embed("Emergency:attending_physician")
v_global = emb.align_to_global(v_local)
```

You can train an alignment matrix on labelled positives and negatives:

```
positives = [
    (emb.embed("attending_physician"),
     emb.embed("doctor")),
    (emb.embed("nurse"),
     emb.embed("registered_nurse")),
]
negatives = [
    (emb.embed("attending_physician"),
     emb.embed("payroll_clerk")),
```

```

]

emb.train_alignment(positives, negatives,
                    epochs=200, lr=0.01, margin=0.5)

# Persist the alignment layer
emb.save_alignment("alignment.npy")
emb.load_alignment("alignment.npy")

```

The alignment functor is the integration mechanism

When two subsystems come from different origins — a merger, an acquisition, a partner API — they use different vocabulary for the same concepts. The alignment functor projects both subsystems' local embeddings into a shared Hilbert space, revealing semantic equivalence. This is the technical mechanism by which CZOI integrates heterogeneous vocabularies: even if two subsystems name the same operation differently, the aligned embeddings still reveal the equivalence. In the paper's post-merger case study, this mechanism discovered 47 semantically equivalent role pairs across two legacy taxonomies — a task that would otherwise have consumed several weeks of manual effort.

12. Adaptive Access Control

The toolkit's signature capability: **permissions that adapt to changing conditions while preserving safety.**

The basic idea

During a surge (a flu outbreak, a trading crash, a supply-chain disruption), the system may need to grant someone a permission they don't normally have. But it must do so without opening a security hole.

```

from czoι import NeuralContribution

def surge_hook(user, operation, zone, base_decision):
    """Grant SeniorNurse the prescribe operation during a surge."""
    if (operation.name == "prescribe"
        and user.has_role("SeniorNurse")
        and zone.properties.get("surge_active", False)):
        return Decision.ALLOW
    return base_decision

builder.permission_engine.set_neural_contribution(
    NeuralContribution(surge_hook)
)

```

The hook receives the base decision (the engine's normal result) and can return:

- The base decision — leave it unchanged.
- `Decision.ALLOW` — elevate.
- `Decision.DENY` — downgrade (lock down).

Toggling adaptation at runtime

```
# Surge starts
builder.root.properties.set("surge_active", True)
builder.permission_engine.invalidate()

# Surge ends
builder.root.properties.set("surge_active", False)
builder.permission_engine.invalidate()
```

Dynamic grants

```
# During surge
hospital.grant(senior_nurse_role, prescribe)

# After surge
hospital.revoke(senior_nurse_role, prescribe)
```

This is often preferable to a hook because the change is visible in the role's `base_permissions` — easy to audit.

Safety guarantees (Theorem 7)

An adaptive update is safe iff:

- 1. It preserves monotonicity — permissions only increase without explicit revocation.
- 2. It satisfies all identity and access constraints.
- 3. It maintains a complete audit trail.

When to use a hook vs a grant

Situation	Use
Temporary elevation during a well-defined event	Grant/revoke — clear boundaries, easy to audit.
Decision depends on complex state	Hook — flexible, no state changes needed.
Time-of-day or attribute-based policy	Hook — re-evaluated on every decision.
Cross-cutting policy applied to many roles	Constraint — expressed once in UniLang.

13. Audit and Observability

The audit trail

```
from czoi import PermissionEngine
```

```
builder.permission_engine = PermissionEngine(
    cache_enabled=True,
    audit_enabled=True,
)
for zone in builder.root.walk():
    zone.set_permission_engine(builder.permission_engine)
```

Every call to `decide` records a `DecisionRecord`:

```
for r in builder.permission_engine.audit[-10:]:
    print(f"{r.timestamp:%H:%M:%S}  {r.user:<12} "
          f"{r.operation:<28} @{{r.zone:<16}} "
          f"→ {r.decision.name:<14} via {r.stage}")
```

Example output:

```
14:23:01  alice          EMR.prescribe          @Emergency      →
ALLOW          via local
14:23:02  bob            EMR.prescribe          @Emergency      →
INCONCLUSIVE   via parent-override
14:23:03  bob            EMR.dispense          @Emergency      →
ALLOW          via local
```

Engine statistics

```
stats = builder.permission_engine.stats()
print(stats)
# {'hits': 43, 'misses': 1180, 'parent_lookups': 4, 'denies': 12}
```

- **hits** — decisions served from cache.
- **misses** — decisions that required engine evaluation.
- **parent_lookups** — number of times the recursive parent lookup fired.
- **denies** — total deny decisions.

Structured logging

```
import logging
import json

logger = logging.getLogger("czoι")

def emit_audit(record):
    logger.info(json.dumps({
        "ts": record.timestamp.isoformat(),
```

```

        "user": record.user,
        "op": record.operation,
        "zone": record.zone,
        "decision": record.decision.name,
        "stage": record.stage,
    )))

    for record in builder.permission_engine.audit:
        emit_audit(record)

```

14. Integrating Heterogeneous Systems

This section is the integration guide. It shows how to wrap legacy systems as adapter zones, federate independent subsystems under shared governance, discover semantically equivalent roles across subsystems, and verify that integration is sound.

Why integration is different from composition

In traditional RBAC, integration means writing adapters between systems. Each adapter is bespoke, unverified, and fragile: change either side and the adapter breaks. Integration is $O(n^2)$ in the number of systems and $O(n)$ in maintenance effort per system change.

CZOI takes a different approach. Integration is **composition of CZOI systems under a common parent**. The composition is a formal operation with verifiable properties (the Integration Theorem, §14.6). Adapter zones present a CZOI interface over legacy systems, so composition treats them identically to native subsystems. Adding a new subsystem is $O(1)$ in the number of existing subsystems; changing one subsystem does not affect the others.

Adapter zones

An **adapter zone** is a leaf CZOI subsystem whose internal behaviour is opaque, but whose interface — a set of roles, operations, and constraints — is declared explicitly. It wraps a legacy system, a third-party SaaS platform, or an external partner API.

The adapter protocol

An adapter must implement two methods:

```

class AdapterProtocol:
    def execute(self, operation_name: str, user_name: str,
                kwargs: dict) -> object:
        """Execute the operation against the underlying system."""

    def describe_interface(self) -> dict:
        """Return {operations, roles, properties} for the wrapped
system."""

```

Wrapping a legacy HR system

```

from czoι import AdapterZone, Operation, Role

class LegacyHRAdapter:
    """Adapter for a 20-year-old HR system."""

    def __init__(self, connection_string):
        self.conn = legacy_connect(connection_string)

    def execute(self, operation_name, user_name, kwargs):
        if operation_name == "LegacyHR.view_employee":
            return self.conn.read_employee(kwargs["employee_id"])
        elif operation_name == "LegacyHR.edit_employee":
            return self.conn.update_employee(
                kwargs["employee_id"], kwargs["changes"],
            )
        raise ValueError(f"Unknown operation: {operation_name}")

    def describe_interface(self):
        return {
            "operations": ["view_employee", "edit_employee"],
            "roles": ["HRViewer", "HREditor"],
        }

# Wrap the legacy system as a CZOI zone
legacy_hr = AdapterZone(
    name="LegacyHR",
    adapter=LegacyHRAdapter("sql://legacy-hr.internal"),
)

# Declare the CZOI-facing interface
view_employee = Operation("view_employee")
edit_employee = Operation("edit_employee")
legacy_hr.expose_operations({
    "view_employee": view_employee,
    "edit_employee": edit_employee,
})

hr_viewer = Role("HRViewer", zone=legacy_hr,
                 base_permissions=[view_employee])
hr_editor = Role("HREditor", zone=legacy_hr,
                 base_permissions=[view_employee, edit_employee])
hr_editor.add_junior(hr_viewer)
legacy_hr.add_role(hr_viewer)
legacy_hr.add_role(hr_editor)

```

From the federation's perspective, `legacy_hr` is indistinguishable from a natively built zone. Its internal state is opaque; only its declared interface participates in permission checks, constraint evaluation, and audit. **The legacy system has not been modified in any way.**

Federating independent subsystems

Once each subsystem is exposed as a zone, federation is a composition operation.

```
from czoι import FederationBuilder

fed = FederationBuilder("MergedHealthAuthority")

# Adopt each subsystem under a common parent
fed.adopt_subsystem(legacy_hr_a, name="LegacyRegionA")
fed.adopt_subsystem(legacy_hr_b, name="LegacyRegionB")
fed.adopt_subsystem(shared_services, name="SharedServices")

# Impose shared governance at the parent level
fed.share_constraint("""
    signature {
        sort User, Role;
        constant HRViewer : Role;
        constant FinanceApprover : Role;
        predicate hasRole(u: User, r: Role);
    }
    forall u: User .
        not (hasRole(u, HRViewer) and hasRole(u, FinanceApprover))
""")

# Wire cross-zone monitoring
fed.add_cross_zone_daemon(FederationHealthDaemon())

# The federation is a full CZOI system
engine = fed.permission_engine
print(engine.decide(alice, view_employee, legacy_hr_a).name)
```

The federation is a CZOI system. It has a permission engine, a constraint manager, a daemon manager, and embeddings — all shared with its subsystems, exactly as in a native CZOI tree.

Discovering role equivalence across subsystems

In a merger, the two legacy systems likely have semantically equivalent roles under different names. The alignment functor discovers these automatically.

```
def discover_role_equivalences(zone_a, zone_b, threshold=0.75):
    """Find semantically equivalent roles across two subsystems."""
    equivalences = []
    for role_a in zone_a.roles.values():
        vec_a = emb.embed_role(role_a)
        for role_b in zone_b.roles.values():
            vec_b = emb.embed_role(role_b)
            sim = emb.similarity(vec_a, vec_b)
            if sim > threshold:
                equivalences.append((role_a, role_b, sim))
```

```

return sorted(equivalences, key=lambda x: -x[2])

# Find equivalent roles across the two legacy regions
equivalences = discover_role_equivalences(legacy_hr_a, legacy_hr_b)
for role_a, role_b, sim in equivalences[:5]:
    print(f"{role_a.name} ↔ {role_b.name}: {sim:.3f}")

```

The discovered equivalences are not automatically applied — a human reviewer approves them, typically via a mapping table that becomes part of the cross-zone integration relation (i). This preserves the human-in-the-loop guarantee that enterprise integration demands.

Cross-zone integration relations

Two sibling zones compose via a **cross-zone integration relation** — a 4-tuple $(\gamma, C, \delta, \epsilon)$:

- γ — a set of inter-zone role mappings that transfer permissions between subsystems.
- C — a set of shared constraints (access, identity, trigger, goal) that both subsystems must respect.
- δ — a set of cross-zone daemons that monitor the interaction between the two subsystems.
- ϵ — a cross-zone embedding alignment that projects both subsystems' vocabularies into a shared space.

```

# Add an integration relation between two sibling subsystems
fed.integrate(
    from_zone=legacy_hr_a,
    to_zone=legacy_hr_b,
    gamma=[
        # Maps: HRViewer@A ≈ HRViewer@B (weight 1.0)
        (legacy_hr_a.roles["HRViewer"],
         legacy_hr_b.roles["HRViewer"],
         1.0),
    ],
    shared_constraints=[
        """forall u: User .
           not (hasRole(u, HRViewer) and hasRole(u, FinanceApprover))""",
    ],
    cross_zone_daemons=[CrossRegionDriftDaemon()],
    embedding_alignment=True,
)

```

The Integration Theorem

Federation is formally sound. The Integration Theorem (Theorem 5 in the paper) guarantees that:

Let S_1 and S_2 be two independent, soundly integrated CZOI systems with no shared operations. Then the composed federation $S = S_1 \oplus S_2$ under a new common parent satisfies:

1. S is a well-formed CZOI system.
2. Every identity constraint of S_1 and S_2 holds in S .
3. Every access constraint of S_1 and S_2 holds in S .

4. Any additional constraint on the parent is enforced for both subsystems.

The corollary applies directly to mergers:

Corollary (Post-Merger Integration Guarantee). In a post-merger scenario where two legacy organizations are integrated under a new parent authority, if both legacy systems are soundly integrated CZOI systems, the merger preserves each legacy system's security guarantees while enabling explicit cross-system policy at the authority level.

Three integration patterns

Every realistic federation is a composition of three fundamental patterns:

Pattern	Categorical name	Use case
Parallel integration	Product (×)	Post-merger integration, multi-tenant SaaS, federated identity
Alternative integration	Coproduct (+)	A/B testing, blue-green deployment, geographic failover
Governed integration	Exponential (→)	Parent company governs subsidiary, regulator supervises regulated entity

End-to-end integration example

Let's federate a legacy HR system and a new finance system into a single federated organization.

```
from czoi import (
    AdapterZone, CZOABuilder, FederationBuilder, Operation, Role, User,
)

# ---- 1. Wrap the legacy HR system -----
legacy_hr = AdapterZone(name="LegacyHR", adapter=MyHRAdapter())
legacy_hr.expose_operations({
    "view_employee": Operation("view_employee"),
})
legacy_hr.add_role(Role("HRViewer", zone=legacy_hr,
                       base_permissions=
[list(legacy_hr.operations.values())[0]]))

# ---- 2. Build the new finance system -----
finance_builder = CZOABuilder("Finance")
finance_app = Application("FinanceApp", zone=finance_builder.root)
approve = finance_app.add_operation(Operation("approve_invoice"))
finance_builder.root.add_application(finance_app)
finance_builder.root.add_role(
    Role("FinanceApprover", zone=finance_builder.root,
         base_permissions=[approve])
)
```

```
# ---- 3. Federate under a shared parent -----
fed = FederationBuilder("MergedCorp")
fed.adopt_subsystem(legacy_hr, name="LegacyHR")
fed.adopt_subsystem(finance_builder, name="Finance")

# ---- 4. Share a separation-of-duty constraint -----
fed.share_constraint("""
    signature {
        sort User, Role;
        constant HRViewer : Role;
        constant FinanceApprover : Role;
        predicate hasRole(u: User, r: Role);
    }
    forall u: User .
        not (hasRole(u, HRViewer) and hasRole(u, FinanceApprover))
""")

# ---- 5. Create users in the federation -----
alice = User("alice", roles={"HRViewer"})
fed.root.add_user(alice)
legacy_hr.add_user(alice)

bob = User("bob", roles={"FinanceApprover"})
fed.root.add_user(bob)
finance_builder.root.add_user(bob)

# ---- 6. The federation is a CZOI system -----
engine = fed.permission_engine
print(engine.decide(alice, list(legacy_hr.operations.values())[0],
                    legacy_hr).name) # ALLOW
print(engine.decide(bob, approve, finance_builder.root).name) # ALLOW

# A user with both roles would be denied by the shared SoD constraint.
charlie = User("charlie", roles={"HRViewer", "FinanceApprover"})
fed.root.add_user(charlie)
legacy_hr.add_user(charlie)
finance_builder.root.add_user(charlie)
print(engine.decide(charlie, approve, finance_builder.root).name) # DENY
```

Cross-zone monitoring

Cross-zone daemons observe subsystem interactions and emit into the federation's common ancestor:

```
from czoι import Daemon, DaemonSignal

class VocabularyDriftDaemon(Daemon):
    """Detects when two subsystems' role vocabularies diverge."""

    def __init__(self, zone_a, zone_b, threshold=0.2, parent=None):
        super().__init__("VocabularyDrift", parent=parent, interval=60.0)
        self.zone_a = zone_a
```

```

        self.zone_b = zone_b
        self.threshold = threshold

    def monitor(self):
        # Compare current role sets
        roles_a = set(self.zone_a.roles.keys())
        roles_b = set(self.zone_b.roles.keys())
        drift = len(roles_a.symmetric_difference(roles_b)) \
            / max(len(roles_a | roles_b), 1)
        if drift > self.threshold:
            self.emit_signal(
                DaemonSignal.STATE_WARNING,
                {"event": "vocabulary_drift", "drift": drift},
            )

class FederationHealthDaemon(Daemon):
    """Root daemon: aggregates cross-zone health signals."""

    def __init__(self, parent=None):
        super().__init__("FederationHealth", parent=parent, interval=30.0)
        self.drift_events = []
        self.latency_events = []

    def on_signal(self, signal, payload, source=None):
        if payload.get("event") == "vocabulary_drift":
            self.drift_events.append((source.name, payload))

fed.add_cross_zone_daemon(FederationHealthDaemon())
fed.add_cross_zone_daemon(
    VocabularyDriftDaemon(legacy_hr_a, legacy_hr_b),
)

```

Verifying integration soundness

You can verify integration soundness programmatically:

```

from czoι.federation import verify_integration

report = verify_integration(fed)
print(report.summary())
# Integration Report
# -----
# Well-formedness           : OK
# Identity preservation    : OK
# Access preservation      : OK
# Constraint flow          : OK
# Cross-zone daemons       : OK
# Audit completeness       : OK

```

If any property is violated, the report identifies the specific subsystem and constraint involved.

Real-world impact

In the paper's post-merger case study, a CZOI federation formed from two legacy health authorities achieved a **61% reduction in integration time**, **100% elimination of duplicate role definitions**, and **zero modifications to the legacy systems**. Both legacy systems retained their compliance certifications, and cross-region specialist consults — impossible in the pre-merger state — became available immediately.

15. Persistence and Web Frameworks

The toolkit is persistence-agnostic. Zones are runtime objects; persistence is a separate concern.

The pattern

1. **Runtime** — CZOI zones, roles, operations.
2. **Persistence** — ordinary ORM models that reference zones by name.
3. **Service** — glue code that reads persisted data, drives the runtime, writes results.

Example: Django

```
# models.py – plain Django models, nothing inheriting from Zone
from django.db import models

class Substation(models.Model):
    name = models.CharField(max_length=64, unique=True)
    voltage_kv = models.FloatField()
    capacity_mw = models.FloatField()

class SensorReading(models.Model):
    substation = models.ForeignKey(Substation, on_delete=models.CASCADE)
    timestamp = models.DateTimeField(auto_now_add=True, db_index=True)
    current_a = models.FloatField()
    voltage_kv = models.FloatField()
```

```
# runtime.py – build the CZOI runtime
from czoι import CZOABuilder

def build_runtime():
    builder = CZOABuilder("Grid")
    for sub_row in Substation.objects.all():
        zone = builder.add_zone(sub_row.name, parent=builder.root)
        zone.properties.set("voltage_kv", sub_row.voltage_kv)
        zone.properties.set("capacity_mw", sub_row.capacity_mw)
    return builder
```

Example: FastAPI

```

from fastapi import FastAPI, Depends, HTTPException
from czoι import Decision

app = FastAPI()
service = GridService()

def current_user(token: str) -> str:
    # ... extract user name from token ...
    return "alice"

@app.post("/dispatch/{operation}")
def dispatch(operation: str, user: str = Depends(current_user)):
    decision = service.dispatch(user, operation)
    if decision is not Decision.ALLOW:
        raise HTTPException(403, detail=decision.name)
    return {"status": "ok"}

```

Persistence in a federation

In a federation, each subsystem can use its own persistence strategy — a legacy database, a modern ORM, a document store. The federation itself only persists:

- The integration relations (γ , C , δ , ϵ) between subsystems.
- The shared constraints imposed at the parent level.
- The audit trail of cross-subsystem decisions.
- The role equivalence mappings discovered by the alignment functor.

This is what makes federation lightweight: the subsystems keep their own data, and the federation only persists the composition metadata.

16. Complete Worked Examples

Two worked examples, each demonstrating a different aspect of CZOI. The first is a native hospital system exercising every feature; the second is a post-merger federation integrating two legacy health authorities.

Example A — A native hospital system

A regional health authority manages multiple hospitals. Each hospital has an Emergency department. During a flu outbreak, senior nurses need temporary authority to prescribe.

```

from czoι import (
    Application, CZOABuilder, Daemon, DaemonSignal, Decision,
    Operation, Predictor, Role, User,
)

builder = CZOABuilder("RegionalHealthAuthority")

region = builder.root

```

```

hosp_a = builder.add_zone("CityHospitalA", parent=region)
emerg_a = builder.add_zone("Emergency", parent=hosp_a, atomic=True)
icu_a = builder.add_zone("ICU", parent=hosp_a, atomic=True)

# Operations and roles
app = Application("EMR", zone=hosp_a)
prescribe = app.add_operation(Operation("prescribe"))
dispense = app.add_operation(Operation("dispense"))
view = app.add_operation(Operation("view_patient"))
hosp_a.add_application(app)

attending = Role("AttendingPhysician", zone=hosp_a,
                 base_permissions=[prescribe, view])
senior_nurse = Role("SeniorNurse", zone=hosp_a,
                    base_permissions=[dispense, view])
nurse = Role("Nurse", zone=hosp_a,
              base_permissions=[dispense, view])
senior_nurse.add_junior(nurse)

hosp_a.add_role(attending)
hosp_a.add_role(senior_nurse)
hosp_a.add_role(nurse)

# Users
def register_along_path(user, zone):
    for z in zone.ancestry():
        if user.name not in z.users:
            z.add_user(user)

alice = User("alice", roles={"AttendingPhysician"})
bob = User("bob", roles={"SeniorNurse"})
for u in [alice, bob]:
    register_along_path(u, emerg_a)
    register_along_path(u, icu_a)

# Constraints
builder.add_access_constraint("""
    signature {
        sort User;
        predicate prescribe(u: User);
        predicate dispense(u: User);
    }
    forall u: User . not (prescribe(u) and dispense(u))
""")

# Neural component: sepsis model
import numpy as np
sepsis_model = Predictor("sepsis", threshold=0.85)
sepsis_model.fit(
    X=np.array([[0.6, 0.4, 0.2], [0.9, 0.8, 0.9]]),
    y=np.array([0.0, 1.0]),
    epochs=500,
)
hosp_a.add_neural("sepsis", sepsis_model)

```

```

# Daemons
class SurgeDaemon(Daemon):
    def __init__(self, hospital, parent=None):
        super().__init__("SurgeDaemon", parent=parent, interval=5.0)
        self.hospital = hospital
        self._active = False

    def monitor(self):
        arrivals = self.hospital.properties.get("recent_arrivals", 0)
        if not self._active and arrivals > 100:
            self._active = True
            self.emit_signal(
                DaemonSignal.STATE_CRITICAL,
                {"event": "surge_started", "arrivals": arrivals},
            )

class DirectorDaemon(Daemon):
    def __init__(self, hospital, ops, parent=None):
        super().__init__("Director", parent=parent, interval=1.0)
        self.hospital = hospital
        self.ops = ops
        self.surge_active = False

    def on_signal(self, signal, payload, source=None):
        if payload.get("event") == "surge_started" and not self.surge_active:
            self.surge_active = True
            senior = self.hospital.roles["SeniorNurse"]
            self.hospital.grant(senior, self.ops["prescribe"])

director = DirectorDaemon(hosp_a, {"prescribe": prescribe})
surge = SurgeDaemon(hosp_a, parent=director)
builder.add_daemon(director)
builder.add_daemon(surge)

# Run
engine = builder.permission_engine
print("Normal:", engine.decide(bob, prescribe, hosp_a).name)    #
INCONCLUSIVE

hosp_a.properties.set("recent_arrivals", 200)
for _ in range(10):
    builder.daemon_manager.tick()

print("During surge:", engine.decide(bob, prescribe, hosp_a).name)    #
ALLOW

```

Example B — A post-merger federation

Two legacy health authorities are integrated under a new parent authority. Each retains its own legacy EMR; a shared services zone provides cross-region workflows.

```

from czoι import (
    AdapterZone, CZOABuilder, FederationBuilder, Operation, Role, User,
)

# ---- 1. Wrap the two legacy EMRs as adapter zones -----
region_a = AdapterZone(name="LegacyRegionA", adapter=RegionAAdapter())
region_a.expose_operations({
    "view_patient": Operation("view_patient"),
    "order_consult": Operation("order_consult"),
})
region_a.add_role(Role("AttendingPhysician", zone=region_a,

base_permissions=list(region_a.operations.values()))

region_b = AdapterZone(name="LegacyRegionB", adapter=RegionBAdapter())
region_b.expose_operations({
    "view_patient": Operation("view_patient"),
    "order_consult": Operation("order_consult"),
})
region_b.add_role(Role("ConsultingSpecialist", zone=region_b,

base_permissions=list(region_b.operations.values()))

# ---- 2. Build a new SharedServices zone -----
shared_builder = CZOABuilder("SharedServices")
shared_app = Application("SharedApp", zone=shared_builder.root)
cross_refer = shared_app.add_operation(Operation("cross_refer"))
shared_builder.root.add_application(shared_app)
shared_builder.root.add_role(
    Role("CrossReferrer", zone=shared_builder.root,
        base_permissions=[cross_refer])
)

# ---- 3. Federate under a new parent authority -----
fed = FederationBuilder("MergedHealthAuthority")
fed.adopt_subsystem(region_a, name="LegacyRegionA")
fed.adopt_subsystem(region_b, name="LegacyRegionB")
fed.adopt_subsystem(shared_builder, name="SharedServices")

# ---- 4. Shared compliance policy at the parent -----
fed.share_constraint("""
    signature {
        sort User;
        predicate prescribe(u: User);
        predicate dispense(u: User);
    }
    forall u: User . not (prescribe(u) and dispense(u))
""")

# ---- 5. Cross-zone integration relation -----
fed.integrate(
    from_zone=region_a,

```

```

    to_zone=region_b,
    gamma=[
        (region_a.roles["AttendingPhysician"],
         region_b.roles["ConsultingSpecialist"],
         0.8),
    ],
    shared_constraints=[
        # Both regions must respect the shared privacy policy
        """forall u: User . not (prescribe(u) and dispense(u))""",
    ],
    cross_zone_daemons=[VocabularyDriftDaemon(region_a, region_b)],
    embedding_alignment=True,
)

# ---- 6. Users and decisions -----
engine = fed.permission_engine

alice = User("alice", roles={"AttendingPhysician"})
fed.root.add_user(alice)
region_a.add_user(alice)

bob = User("bob", roles={"ConsultingSpecialist"})
fed.root.add_user(bob)
region_b.add_user(bob)

# Alice can view patients in Region A (native access).
print("A:", engine.decide(alice, region_a.operations["view_patient"],
                          region_a.name)    # ALLOW

# Alice cannot view patients in Region B (no cross-region mapping).
print("B:", engine.decide(alice, region_b.operations["view_patient"],
                          region_b.name)    # DENY

# After adding a cross-region y mapping, Alice gains access.
fed.integrate(
    from_zone=region_a,
    to_zone=region_b,
    gamma=[(region_a.roles["AttendingPhysician"],
            region_b.roles["ConsultingSpecialist"],
            1.0)],
)
# Alice's AttendingPhysician role now transfers to the consulting role.
print("B after mapping:",
      engine.decide(alice, region_b.operations["view_patient"],
                    region_b.name)    # ALLOW

```

What these examples demonstrate

- **Recursive zones** — region → hospital → emergency/ICU, and region → legacy EMR.
- **Seniority inheritance** — SeniorNurse inherits Nurse's permissions.
- **Separation of duty** — no user can both prescribe and dispense.
- **Neural components** — sepsis predictor trained on synthetic vitals.

- **Hierarchical daemons** — SurgeDaemon signals DirectorDaemon, which dynamically grants a permission.
- **Adapter zones** — legacy EMRs wrapped without modification.
- **Federation** — two legacy regions plus a new shared service, composed under a shared parent.
- **Cross-zone integration** — γ mappings enable cross-region access; shared constraints impose unified policy.
- **Embedding alignment** — semantic equivalence between roles discovered automatically.
- **Integration soundness** — the composed system preserves each subsystem's invariants.

17. Cookbook

Find all zones where a user has a role

```
def zones_with_role(user, role_name):
    return [z for z in builder.root.walk()
            if role_name in user.roles and role_name in z.roles]

print(zones_with_role(bob, "SeniorNurse"))
```

List every operation a user can perform

```
def accessible_operations(user, zone):
    engine = builder.permission_engine
    return [op.qualified_name for op in zone.operations.values()
            if engine.decide(user, op, zone) is Decision.ALLOW]

print(accessible_operations(alice, emergency))
```

Find the least-senior role that can perform an operation

```
def least_role(zone, operation):
    candidates = [r for r in zone.roles.values()
                  if operation in r.base_permissions]
    if not candidates:
        return None
    return min(candidates, key=lambda r: len(r.junior_roles))

print(least_role(hosp_a, prescribe).name)
```

Snapshot the system to JSON

```
import json
```

```
def snapshot(builder):
    def zone_repr(zone):
        return {
            "name": zone.name,
            "roles": list(zone.roles.keys()),
            "operations": list(zone.operations.keys()),
            "users": list(zone.users.keys()),
            "properties": zone.properties.as_dict(),
            "children": [zone_repr(c) for c in zone.zones.values()],
        }
    return zone_repr(builder.root)

print(json.dumps(snapshot(builder), indent=2))
```

Trace a permission decision step by step

```
def trace_decision(engine, user, operation, zone):
    print(f"Trace: {user.name} → {operation.qualified_name} @ {zone.name}")
    for z in [zone] + zone.ancestry()[::-1]:
        decision = engine.evaluate_local(user, operation, z)
        print(f"  @{z.name}: {decision.name}")
        if decision is not Decision.INCONCLUSIVE:
            break
```

Block all operations for a user (security lockdown)

```
def lockdown(user, zone):
    for role_name in list(user.roles):
        role = zone.roles.get(role_name)
        if role is None:
            continue
        for op in list(role.base_permissions):
            zone.revoke(role, op)
    user.attributes.set("locked_down", True)

lockdown(bob, hosp_a)
```

Round-trip a zone tree from a config file

```
# zones.yaml
# zones:
#   - name: Hospital
#     children:
#       - name: Emergency
#       - name: ICU
#       - name: Surgery
```

```
import yaml

def from_yaml(builder, path):
    data = yaml.safe_load(open(path))
    def build(specs, parent):
        for spec in specs:
            zone = builder.add_zone(spec["name"], parent=parent)
            if spec.get("children"):
                build(spec["children"], parent=zone)
    build(data["zones"], builder.root)
```

Discover semantically equivalent roles across subsystems

```
from czoι import EmbeddingService

emb = EmbeddingService(dimension=128, use_transformer=True)

def find_equivalences(zone_a, zone_b, threshold=0.75):
    results = []
    for role_a in zone_a.roles.values():
        vec_a = emb.embed_role(role_a)
        for role_b in zone_b.roles.values():
            vec_b = emb.embed_role(role_b)
            sim = emb.similarity(vec_a, vec_b)
            if sim > threshold:
                results.append((role_a.name, role_b.name, sim))
    return sorted(results, key=lambda x: -x[2])
```

Wrap a legacy system as an adapter zone

```
from czoι import AdapterZone, Operation, Role

class MyLegacyAdapter:
    def __init__(self, conn_string):
        self.conn = legacy_connect(conn_string)

    def execute(self, operation_name, user_name, kwargs):
        # Translate CZOI operation to legacy call
        return self.conn.call(operation_name, kwargs)

    def describe_interface(self):
        return {
            "operations": ["view_record", "edit_record"],
            "roles": ["Viewer", "Editor"],
        }

legacy = AdapterZone(name="MyLegacy", adapter=MyLegacyAdapter(conn))
legacy.expose_operations({
```

```

    "view_record": Operation("view_record"),
    "edit_record": Operation("edit_record"),
})
legacy.add_role(Role("Viewer", zone=legacy,
                    base_permissions=[legacy.operations["view_record"]]))

```

Federate two subsystems

```

from czoil import FederationBuilder

fed = FederationBuilder("MergedCorp")
fed.adopt_subsystem(subsystem_a, name="A")
fed.adopt_subsystem(subsystem_b, name="B")
fed.share_constraint(""...")
fed.integrate(from_zone=subsystem_a, to_zone=subsystem_b,
              gamma=[(role_a, role_b, 1.0)])
fed.add_cross_zone_daemon(FederationHealthDaemon())

```

18. Troubleshooting

ZoneContainmentError: User X must be affiliated with parent Y

You tried to register a user in a child zone without registering them in the parent first.

```

# Wrong
emergency.add_user(alice)

# Right
for z in emergency.ancestry():
    z.add_user(alice)

```

TypeError: AtomicZone cannot have child zones

```

# Wrong
leaf = builder.add_zone("Leaf", parent=parent, atomic=True)
leaf.add_zone(builder.add_zone("Child")) # raises

# Right
leaf = builder.add_zone("Leaf", parent=parent) # composite
leaf.add_zone(builder.add_zone("Child"))

```

UniLogBridgeError: The unilog-toolkit is required

```
pip install unilog-toolkit
```

UniLangSyntaxError: Unexpected character X

Check for: missing semicolons after signature declarations; comments using `//` instead of `#` or `%`; Unicode operators mixed with ASCII mid-expression.

Decision.INCONCLUSIVE when you expect DENY

INCONCLUSIVE means "no covering role." **DENY** means "covering role, but a constraint blocked it." If you want boolean semantics:

```
allowed = engine.decide(user, op, zone) is Decision.ALLOW
```

Cache staleness

```
# Ensure you used the zone-level method
hospital.grant(role, operation) # invalidates cache
# Not:
role.grant(operation)          # does NOT invalidate cache

# Or invalidate manually:
builder.permission_engine.invalidate()
```

Daemon not firing

Check: (1) you called `builder.add_daemon(daemon)`; (2) `daemon.enabled` is `True`; (3) `daemon.interval` is set appropriately; (4) you're calling `tick()` or `run()`.

```
print(daemon.enabled, daemon.interval)
print([d.name for d in builder.daemon_manager.daemons])
```

Neural model predictions are nonsense

Most commonly: features aren't normalised (z-score everything before fitting); training data has no signal; or threshold is miscalibrated.

```
print(f"Mean positive score: {X_pos.mean(axis=0)}")
print(f"Mean negative score: {X_neg.mean(axis=0)}")
# The two should differ substantially.
```

Integration-specific issues

FederationError: Subsystems have conflicting operations

Two subsystems are exposing operations with the same qualified name. Rename one or use adapter zones to expose them under distinct prefixes.

```
# Rename in the adapter zone
legacy_hr.expose_operations({
    "LegacyHR.view_employee": Operation("view_employee"),
    # Avoid: "view_employee" alone would collide with another subsystem
})
```

Integration error: role equivalence not discovered

The embedding alignment threshold may be too high, or the two subsystems' vocabularies may be too different for the current embedding model. Lower the threshold, or train the alignment functor with domain-specific positives and negatives.

```
emb.train_alignment(
    positives=[(emb.embed("attending"), emb.embed("physician"))],
    negatives=[(emb.embed("attending"), emb.embed("payroll"))],
)
```

Integration validation failed: identity constraint violation

The composed system violates a subsystem's identity constraints. This typically means the shared constraints at the parent level are incompatible with a subsystem's local constraints. Use `verify_integration(fed)` to identify the specific constraint.

```
from czoι.federation import verify_integration
report = verify_integration(fed)
print(report.details())
```

19. Best Practices

Structure

- **Model the organisation, not the code.** If the HR department is a zone, so be it — even if it only has two users.
- **Use atomic zones generously.** Any zone without children should be `atomic=True`.
- **Name operations with `verb_noun`.** `view_patient`, `process_payroll`, not `viewPatient` or `VP`.

Roles

- **One role per job function.** If you find yourself writing `role_that_can_do_X_and_Y`, use a seniority relation instead.
- **Prefer seniority over duplication.**
- **Keep role hierarchies shallow.** Depth 3 is usually enough.

Operations

- **Atomic operations, not composite actions.** `submit_grade` is atomic; `finalise_semester` is not.
- **Fine granularity for security-critical operations.** Prefer `edit_salary`, `edit_address`, `edit_bank_account`.
- **Application grouping is for humans, not for permissions.**

Constraints

- **Encode policy, not implementation.** "A user cannot both prescribe and dispense" is policy.
- **One constraint per rule.**
- **Test constraints in isolation.**

Daemons

- **One responsibility per daemon.**
- **Signal, don't act.** A daemon's `monitor()` should emit signals; the parent's `on_signal()` should act.
- **Use per-daemon intervals.**

Neural components

- **Normalise features.** Always.
- **Fit on normal data only** for anomaly detection.
- **Version your models.** Record which model produced which decision in the audit trail.

Testing

- **Test with a fresh builder per test.**
- **Test `ALLOW`, `DENY`, and `INCONCLUSIVE` separately.**
- **Test constraints explicitly.**

Integration

- **Start with adapter zones, not refactoring.** Wrap legacy systems as-is. This preserves their certifications and gives you a working baseline before any internal change.
- **Use the alignment functor to discover role equivalence, but review manually.** Automated discovery finds candidates; a human decides which mappings to apply.
- **Prefer shared constraints over duplicated ones.** If two subsystems need the same rule, share it at the parent level rather than duplicating it.
- **Add cross-zone daemons from day one.** They detect integration drift before it becomes an incident.
- **Verify integration soundness programmatically.** Run `verify_integration(fed)` after every integration change.
- **Document the integration relation.** The $(\gamma, C, \delta, \epsilon)$ tuple is the contract between subsystems; treat it like an API contract.

- **Preserve local autonomy explicitly.** Don't impose a shared constraint unless it's genuinely organization-wide. Local autonomy is not a cost to minimize; it's a property to protect.
- **Test integration across subsystem boundaries.** Every cross-zone decision should have at least one test that exercises the path in both directions.

Performance

- **Cache is your friend.** Keep `cache_enabled=True` and monitor `stats()["hits"]`.
- **Avoid wide fan-outs.** Prefer smaller roles with seniority relations.
- **Deep trees are fine.** Recursion depth adds ~0.02 ms per level.
- **Cross-zone decisions add ~0.1 ms per subsystem boundary crossed.** For deeply federated systems, cache frequently-used cross-zone decisions at the federation level.

20. API Reference

CZOABuilder

Method	Description
<code>CZOABuilder(name)</code>	Create a builder with a root composite zone
<code>.root</code>	The root <code>CompositeZone</code>
<code>.add_zone(name, parent=None, atomic=False)</code>	Add a zone; parent defaults to root
<code>.permission_engine</code>	The shared <code>PermissionEngine</code>
<code>.constraint_manager</code>	The shared <code>ConstraintManager</code>
<code>.daemon_manager</code>	The shared <code>DaemonManager</code>
<code>.embedding_service</code>	The shared <code>EmbeddingService</code>
<code>.add_access_constraint(unilang)</code>	Register an access constraint
<code>.add_identity_constraint(unilang)</code>	Register an identity constraint
<code>.add_trigger_constraint(unilang)</code>	Register a trigger constraint
<code>.add_goal_constraint(unilang)</code>	Register a goal constraint
<code>.register_predicate(name, fn)</code>	Register a custom predicate for constraints
<code>.add_daemon(daemon)</code>	Register a daemon with the root and manager

FederationBuilder ✨ (new)

Method	Description
<code>FederationBuilder(name)</code>	Create a federation with a fresh parent zone

Method	Description
<code>.adopt_subsystem(subsystem, name=None)</code>	Adopt a subsystem (native CZOI, adapter zone, or federation) under the parent
<code>.share_constraint(unilang)</code>	Impose a shared constraint at the parent level
<code>.integrate(from_zone, to_zone, gamma=[], shared_constraints=[], cross_zone_daemons=[], embedding_alignment=False)</code>	Define a cross-zone integration relation
<code>.add_cross_zone_daemon(daemon)</code>	Register a daemon that observes subsystem interactions
<code>.root</code>	The federation's root zone
<code>.permission_engine</code>	The shared <code>PermissionEngine</code>

AdapterZone ✨ (new)

Method	Description
<code>AdapterZone(name, parent=None, adapter=None)</code>	Wrap a legacy or external system
<code>.adapter</code>	The adapter instance (implements <code>AdapterProtocol</code>)
<code>.expose_operations(dict)</code>	Declare the CZOI-facing operations
<code>.delegate(operation, user, **kwargs)</code>	Forward a call to the underlying adapter

ZoneBase

Attribute / Method	Description
<code>.name</code>	Zone name
<code>.parent</code>	Parent zone (None for root)
<code>.zones</code>	<code>dict[str, ZoneBase]</code> of children
<code>.roles</code>	<code>dict[str, Role]</code>
<code>.users</code>	<code>dict[str, User]</code>
<code>.applications</code>	<code>dict[str, Application]</code>
<code>.operations</code>	<code>dict[str, Operation]</code>
<code>.neural</code>	<code>dict[str, Any]</code>
<code>.embeddings</code>	The embedding service

Attribute / Method	Description
<code>.constraints</code>	The constraint manager
<code>.permission_engine</code>	The permission engine
<code>.daemons</code>	<code>list[Daemon]</code>
<code>.properties</code>	<code>PropertyStore</code>
<code>.add_zone(child)</code>	Register a child (raises on atomic)
<code>.add_role(role)</code>	Register a role; propagates to children
<code>.add_user(user)</code>	Register a user (enforces containment)
<code>.add_application(app)</code>	Register an application and its operations
<code>.add_operation(op)</code>	Register a bare operation
<code>.add_neural(name, component)</code>	Register a neural component
<code>.add_daemon(daemon)</code>	Register a daemon with this zone
<code>.grant(role, operation)</code>	Grant and invalidate cache
<code>.revoke(role, operation)</code>	Revoke and invalidate cache
<code>.walk()</code>	Iterate this zone and all descendants
<code>.ancestry()</code>	List from root down to this zone
<code>.depth()</code>	Depth from the root (root is 0)
<code>.is_ancestor_of(other)</code>	True if <code>self</code> is an ancestor of <code>other</code>

Role

Method	Description
<code>Role(name, zone=None, base_permissions=[], attributes={})</code>	Construct
<code>.name, .zone</code>	Identity
<code>.base_permissions</code>	<code>set[Operation]</code>
<code>.junior_roles</code>	<code>set[Role]</code>
<code>.grant(op) / .revoke(op)</code>	Modify base permissions directly (no cache invalidation)
<code>.add_junior(role)</code>	Declare this role senior to <code>role</code>
<code>.qualified_name</code>	<code>"<zone>:<name>"</code>

User

Method	Description
<code>User(name, roles=set(), attributes={}, credentials={})</code>	Construct
<code>.name, .roles, .attributes, .credentials</code>	State
<code>.has_role(name)</code>	Membership test

Operation, Application

Method	Description
<code>Operation(name, application=None, properties={})</code>	Construct
<code>.qualified_name</code>	"<app>.<name>"
<code>Application(name, zone=None, properties={})</code>	Construct
<code>.add_operation(op)</code>	Register an operation (sets <code>.application</code>)

PermissionEngine

Method	Description
<code>PermissionEngine(cache_enabled=True, audit_enabled=False)</code>	Construct
<code>.decide(user, operation, zone)</code>	Returns Decision
<code>.evaluate_local(user, operation, zone)</code>	Single-stage decision (no recursion)
<code>.invalidate()</code>	Clear the decision cache
<code>.stats()</code>	Dict with hits , misses , parent_lookups , denies
<code>.audit</code>	List of DecisionRecord (if enabled)
<code>.set_neural_contribution(contribution)</code>	Attach a NeuralContribution hook

Decision

Enum: `Decision.ALLOW`, `Decision.DENY`, `Decision.INCONCLUSIVE`.

ConstraintManager

Method	Description
<code>.add(kind, source, name=None)</code>	Parse and register a UniLang constraint
<code>.add_access(source) / .add_identity(...) / .add_trigger(...) / .add_goal(...)</code>	Convenience methods

Method	Description
<code>.register_predicate(name, fn)</code>	Custom predicate
<code>.check_all(request=None)</code>	Evaluate all constraints; return <code>dict[kind, bool]</code>
<code>.check_zone(zone=None, request=None)</code>	Same, scoped to a zone
<code>.is_satisfied(user, operation, zone)</code>	Hook used by the permission engine
<code>.attach(zone)</code>	Bind to the root zone (called automatically)

Daemon

Method	Description
<code>Daemon(name, parent=None, interval=1.0)</code>	Construct
<code>.monitor()</code>	Override — periodic work
<code>.on_signal(signal, payload, source=None)</code>	Override — handle child signals
<code>.emit_signal(signal, payload)</code>	Emit upward
<code>.start() / .stop()</code>	Enable / disable
<code>.safe_monitor()</code>	Error-isolated monitor (called by manager)
<code>.register_with(zone)</code>	Called automatically

DaemonSignal

Enum: `STATE_NORMAL`, `STATE_WARNING`, `STATE_CRITICAL`, `REVOKE`, `ESCALATE`.

DaemonManager

Method	Description
<code>DaemonManager(default_interval=1.0, max_workers=4)</code>	Construct
<code>.add(daemon) / .extend(daemons)</code>	Register
<code>.tick()</code>	Synchronous single-pass
<code>await .run(duration=None)</code>	Async loop
<code>.stop()</code>	Stop the loop
<code>.shutdown()</code>	Stop and shut down the executor

Predictor

Method	Description
<code>Predictor(name, fn=None, threshold=0.5, input_dim=None)</code>	Construct
<code>.fit(X, y, epochs=200, lr=0.05)</code>	Train
<code>.predict(features)</code>	<code>float</code> in <code>[0, 1]</code>
<code>.fires(features)</code>	<code>bool</code> (<code>predict >= threshold</code>)

AnomalyDetector

Method	Description
<code>AnomalyDetector(name, input_dim, latent_dim=8, threshold=0.1, seed=0)</code>	Construct
<code>.fit(X, epochs=200, lr=0.05)</code>	Train and calibrate threshold
<code>.score(x)</code>	Reconstruction error (float)
<code>.is_anomalous(x)</code>	<code>bool</code>

RoleMiner

Method	Description
<code>RoleMiner(latent_dim=16, min_cluster_size=3, seed=0, ...)</code>	Construct
<code>.mine(X, operation_names, min_support=0.5)</code>	Returns <code>MiningResult</code>

`MiningResult` fields: `suggested_roles`, `confidence`, `n_clusters`.

EmbeddingService

Method	Description
<code>EmbeddingService(dimension=64, model_name=..., use_transformer=False)</code>	Construct
<code>.embed(text)</code>	Vector for a string
<code>.embed_operation(op) / .embed_role(role)</code>	Entity-specific embeddings
<code>.align_to_global(v)</code>	Project a local vector into the shared space
<code>.train_alignment(positives, negatives, epochs, lr, margin)</code>	Fit the alignment layer

Method	Description
<code>.similarity(a, b)</code>	Cosine similarity
<code>.save_alignment(path) / .load_alignment(path)</code>	Persistence

NeuralContribution

```
NeuralContribution(fn: Callable[[user, operation, zone, base], Decision])
```

Attach with `builder.permission_engine.set_neural_contribution(...)`.

Integration utilities ✨ (new)

```
from czoι.federation import verify_integration, discover_role_equivalences

# Verify that a federation is soundly integrated
report = verify_integration(fed)
print(report.summary())

# Discover role equivalences across two subsystems
equivalences = discover_role_equivalences(zone_a, zone_b,
                                          embedding_service=emb,
                                          threshold=0.75)
```

Exceptions

Exception	Raised when
<code>CZOIError</code>	Base class
<code>ZoneError</code>	Invalid zone operation
<code>ZoneContainmentError</code>	$U_{child} \subseteq U_{parent}$ violated
<code>RoleError</code>	Role definition problem
<code>UserError</code>	User definition problem
<code>ApplicationError</code>	Application problem
<code>PermissionError</code>	Permission calculus violation
<code>ConstraintError</code>	Constraint manager problem
<code>SafetyViolation</code>	Adaptive update violated a safety invariant
<code>UniLogBridgeError</code>	UniLog toolkit unavailable
<code>UniLangSyntaxError</code>	Parse error in UniLang source

Exception	Raised when
<code>UniLangSemanticError</code>	Undefined sort, arity mismatch
<code>UniLangEvaluationError</code>	Runtime evaluation error
<code>FederationError</code> ✨	Federation composition failure (conflicting operations, incompatible constraints, etc.)
<code>AdapterError</code> ✨	Adapter zone contract violation (missing method, unknown operation, etc.)

Getting Help

- **Examples:** the `examples/` directory in the repository, including `legacy_adapter.py` and `post_merger.py`.
- **Tests:** `tests/test_basic.py` covers core behaviour; `tests/test_federation.py` covers integration.
- **Issues:** github.com/hongxueharriswang/czoi-toolkit/issues
- **UniLog:** github.com/hongxueharriswang/unilog-toolkit
- **Tutorial:** the companion [CZOA/CZOI Tutorial](#) teaches the theory and practice, including Chapter 15 on integrating heterogeneous systems.

End of User Guide.